SDD — Módulo de Visualización y Análisis de Grafos en C estándar con Raylib

Proyecto general: Visualizador de Estructuras de Datos en C estándar con Raylib
Módulo: Grafos
Lenguaje: C estándar
Biblioteca gráfica: `raylib.h`
Contexto académico: Asignatura Estructuras de Datos — Ingeniería de Sistemas y Computación — Universidad Tecnológica de Pereira
Versión: 1.0
Autor académico: Francisco Alejandro Medina Aguirre

---

1. Información general del proyecto

1.1 Nombre del módulo

Módulo de Visualización y Análisis de Grafos en C estándar con Raylib.

1.2 Descripción general

Este módulo amplía el visualizador académico de estructuras de datos incorporando el manejo, representación gráfica y análisis algorítmico de **grafos dirigidos, no dirigidos y ponderados**. El sistema debe permitir construir grafos de manera interactiva, representar visualmente vértices y aristas, ejecutar algoritmos clásicos y mostrar simultáneamente:

1. La representación visual del grafo.
2. El código C asociado a la operación o algoritmo seleccionado.
3. La explicación paso a paso del algoritmo.
4. La complejidad temporal y espacial.
5. El resultado obtenido: recorrido, camino mínimo o árbol de expansión mínima.

El módulo debe integrarse con el TAD suministrado:

- `grafo.h`
- `grafo.c`

El TAD define una API pública para grafos usando un tipo opaco `Grafo`, resultados estructurados para recorridos y caminos, y algoritmos como BFS, DFS, Dijkstra, Bellman-Ford, Prim y Kruskal.

1.3 Objetivo del módulo

Diseñar e implementar un módulo gráfico e interactivo que permita visualizar la construcción y ejecución de algoritmos sobre grafos, manteniendo separada la lógica del TAD, la lógica visual y la lógica didáctica, con documentación Doxygen y compatibilidad con GCC y Raylib.

1.4 Alcance

El módulo incluirá las siguientes funcionalidades:

- Crear grafo dirigido o no dirigido.
- Insertar vértices.
- Insertar aristas ponderadas.
- Eliminar vértices.
- Eliminar aristas.
- Consultar orden y tamaño del grafo.
- Consultar grado de entrada y grado de salida.
- Visualizar sucesores y predecesores.
- Ejecutar BFS.
- Ejecutar DFS.
- Ejecutar Dijkstra.
- Ejecutar Bellman-Ford.
- Ejecutar Prim.
- Ejecutar Kruskal.
- Mostrar código, explicación y complejidad asociada.

1.5 Fuera del alcance inicial

No se incluirá en la primera versión:

- Edición avanzada de posiciones mediante arrastre persistente.
- Importación/exportación de grafos en formatos externos como GraphML o JSON.
- Animaciones físicas complejas tipo force-directed layout.
- Grafos multiarista.
- Grafos con etiquetas textuales no numéricas.
- Algoritmos de flujo máximo, coloreo o ciclos hamiltonianos.

---

2. Visión del sistema

2.1 Problema que resuelve

Los algoritmos de grafos suelen ser difíciles de comprender únicamente desde el código fuente, debido a que involucran relaciones no lineales entre vértices, aristas, pesos, estructuras auxiliares y decisiones iterativas. En clase, los estudiantes pueden confundirse al analizar:

- Qué vértices han sido visitados.
- Qué aristas se relajan en un camino mínimo.
- Cómo se forma un árbol de expansión mínima.
- Qué diferencia existe entre BFS y DFS.
- Por qué Dijkstra no acepta pesos negativos.
- Cómo Bellman-Ford detecta ciclos negativos.
- Cómo Prim y Kruskal seleccionan aristas.

El módulo busca conectar la teoría, el código C y la visualización interactiva para facilitar el aprendizaje significativo.

2.2 Usuarios objetivo

Usuario	Descripción
Estudiante	Construye grafos, ejecuta algoritmos y analiza resultados paso a paso.
Docente	Usa el módulo como apoyo visual para explicar algoritmos de grafos.
Monitor académico	Apoya prácticas de laboratorio y resolución de ejercicios.
Desarrollador académico	Mantiene y extiende el visualizador con nuevos algoritmos.

2.3 Beneficios esperados

- Facilitar la comprensión visual de grafos y algoritmos asociados.
- Relacionar el TAD Grafo en C con su representación gráfica.
- Apoyar el análisis de complejidad algorítmica.
- Promover buenas prácticas de encapsulamiento, modularidad y documentación.
- Proporcionar una herramienta reutilizable para laboratorios y clases.

2.4 Contexto de uso

La aplicación será usada en sesiones de clase, laboratorios de programación, proyectos finales y actividades autónomas del curso de Estructuras de Datos. Debe funcionar como una aplicación de escritorio desarrollada en C estándar con Raylib, preferiblemente compilable desde GCC en Windows.

---

3. Restricciones técnicas

ID	Restricción
RT-01	El núcleo lógico del grafo debe escribirse en C estándar.
RT-02	La interfaz gráfica debe desarrollarse con `raylib.h`.
RT-03	El TAD Grafo no debe depender de Raylib.
RT-04	`grafo.h` debe exponer únicamente la API pública del TAD.
RT-05	`grafo.c` debe encapsular la implementación interna de vértices y aristas.
RT-06	El módulo gráfico debe acceder al grafo únicamente mediante funciones públicas.
RT-07	Todo módulo, estructura y función pública debe documentarse con Doxygen.
RT-08	Las operaciones deben validar punteros nulos, vértices inexistentes, aristas inexistentes y errores de memoria.
RT-09	Los resultados dinámicos de recorridos y caminos deben liberarse con las funciones del TAD.
RT-10	El sistema debe poder compilarse con GCC y Raylib.
RT-11	La visualización debe separar dibujo, entrada del usuario, estado de la aplicación y lógica algorítmica.

ID	Restricción
RT-12	Los algoritmos deben mostrar claramente su complejidad y sus restricciones.

---

4. Arquitectura general del módulo

4.1 Estilo arquitectónico

Se mantiene la arquitectura modular por capas definida para el visualizador general:

```
+-------------------------------------------------------------------+
|                         Capa de Presentación                      |
|          Raylib, paneles, botones, dibujo de vértices/aristas     |
+-------------------------------------------------------------------+
|                         Capa de Control Visual                    |
|       Eventos, selección de algoritmo, selección de vértices      |
+-------------------------------------------------------------------+
|                         Capa Didáctica                            |
|       Código mostrado, trazas, explicación y complejidad          |
+-------------------------------------------------------------------+
|                         Capa de Dominio                           |
|                         grafo.h / grafo.c                         |
+-------------------------------------------------------------------+
|                         Infraestructura                           |
|       Assets, compilación, Doxygen, Makefile, Raylib              |
+-------------------------------------------------------------------+
```

4.2 Principios de diseño

1. **Encapsulamiento:** el módulo visual no accede a `NodoVertice` ni `NodoArista`, ya que son estructuras internas del TAD.
2. **Separación:** los algoritmos se ejecutan desde `grafo.c`; la vista solo interpreta resultados públicos.
3. **Reutilización:** el TAD Grafo puede usarse en consola, pruebas unitarias o interfaz gráfica.
4. **Trazabilidad:** cada acción visual debe tener relación con una operación del TAD.
5. **Didáctica:** cada algoritmo debe mostrar pasos, colores, código y complejidad.
6. **Extensibilidad:** el módulo debe permitir agregar nuevos algoritmos en el futuro.

---

5. Estructura propuesta de carpetas

```
visualizador-ed/
│
├── assets/
│   ├── logo_utp.png
│   ├── logo_isc.png
│   └── fonts/
```

```
│
├── include/
│   ├── grafo.h
│   ├── grafo_view.h
│   ├── grafo_controller.h
│   ├── grafo_layout.h
│   ├── grafo_code_viewer.h
│   ├── grafo_trace.h
│   ├── grafo_state.h
│   └── ui.h
│
├── src/
│   ├── grafo.c
│   ├── grafo_view.c
│   ├── grafo_controller.c
│   ├── grafo_layout.c
│   ├── grafo_code_viewer.c
│   ├── grafo_trace.c
│   ├── grafo_state.c
│   ├── ui.c
│   └── main.c
│
├── tests/
│   ├── test_grafo_basico.c
│   ├── test_grafo_recorridos.c
│   ├── test_grafo_caminos.c
│   └── test_grafo_mst.c
│
├── docs/
│   ├── Doxyfile
│   ├── sdd_modulo_grafos.md
│   └── guia_laboratorio_grafos.md
│
└── Makefile
```

---

6. Modelo de componentes

Componente	Responsabilidad
`grafo.h`	Define la API pública del TAD Grafo.
`grafo.c`	Implementa vértices, aristas, recorridos, caminos mínimos y MST.
`grafo_state.h/c`	Administra el estado visual y didáctico del módulo.
`grafo_view.h/c`	Dibuja vértices, aristas, pesos, flechas y resultados.
`grafo_layout.h/c`	Calcula posiciones de vértices en pantalla.

Componente	Responsabilidad
`grafo_controller.h/c`	Procesa eventos del usuario y llama al TAD.
`grafo_code_viewer.h/c`	Muestra fragmentos de código C asociados a operaciones.
`grafo_trace.h/c`	Genera explicación paso a paso de algoritmos.
`ui.h/c`	Dibuja encabezado, menú, botones y estilos comunes.
`main.c`	Inicializa la aplicación y ejecuta el ciclo principal.

7. Requisitos funcionales

ID	Requisito funcional	Prioridad	Actor
RF-01	El sistema debe permitir crear un grafo dirigido o no dirigido.	Alta	Usuario
RF-02	El sistema debe permitir insertar vértices identificados por números enteros.	Alta	Usuario
RF-03	El sistema debe permitir insertar aristas ponderadas entre vértices existentes.	Alta	Usuario
RF-04	El sistema debe permitir eliminar vértices y sus aristas incidentes.	Media	Usuario
RF-05	El sistema debe permitir eliminar aristas.	Media	Usuario
RF-06	El sistema debe dibujar vértices como nodos circulares.	Alta	Usuario
RF-07	El sistema debe dibujar aristas con líneas o flechas según el tipo de grafo.	Alta	Usuario
RF-08	El sistema debe mostrar el peso de cada arista.	Alta	Usuario
RF-09	El sistema debe mostrar orden y tamaño del grafo.	Media	Usuario
RF-10	El sistema debe consultar grado de entrada y salida de un vértice.	Media	Usuario
RF-11	El sistema debe mostrar sucesores y predecesores de un vértice.	Media	Usuario
RF-12	El sistema debe ejecutar BFS desde un vértice inicial.	Alta	Usuario
RF-13	El sistema debe ejecutar DFS desde un vértice inicial.	Alta	Usuario
RF-14	El sistema debe ejecutar Dijkstra entre un vértice origen y destino.	Alta	Usuario
RF-15	El sistema debe ejecutar Bellman-Ford entre un vértice origen y destino.	Alta	Usuario
RF-16	El sistema debe ejecutar Prim desde un vértice inicial en grafos no dirigidos.	Alta	Usuario
RF-17	El sistema debe ejecutar Kruskal en grafos no dirigidos.	Alta	Usuario
RF-18	El sistema debe mostrar el código C asociado a cada operación o algoritmo.	Alta	Usuario

ID	Requisito funcional	Prioridad	Actor
RF-19	El sistema debe mostrar explicación paso a paso del algoritmo ejecutado.	Alta	Usuario
RF-20	El sistema debe mostrar complejidad temporal y espacial de cada algoritmo.	Alta	Usuario
RF-21	El sistema debe resaltar vértices visitados en recorridos.	Alta	Usuario
RF-22	El sistema debe resaltar aristas seleccionadas en caminos mínimos y MST.	Alta	Usuario
RF-23	El sistema debe mostrar mensajes claros ante errores del TAD.	Alta	Usuario
RF-24	El sistema debe permitir limpiar el grafo actual.	Media	Usuario
RF-25	El sistema debe liberar memoria de resultados dinámicos al cambiar de algoritmo.	Alta	Sistema

8. Requisitos no funcionales

ID	Tipo	Requisito no funcional
RNF-01	Usabilidad	La interfaz debe organizarse en panel visual, panel de código, panel de explicación y panel de resultados.
RNF-02	Rendimiento	La aplicación debe mantener una tasa mínima de 30 FPS en grafos pequeños y medianos.
RNF-03	Mantenibilidad	Los módulos deben estar separados por responsabilidad.
RNF-04	Documentación	Todas las funciones públicas deben tener comentarios Doxygen.
RNF-05	Portabilidad	El sistema debe compilar con GCC y Raylib en Windows.
RNF-06	Robustez	Las operaciones deben manejar errores sin cerrar abruptamente la aplicación.
RNF-07	Seguridad de memoria	Los arreglos dinámicos retornados por el TAD deben liberarse correctamente.
RNF-08	Extensibilidad	Debe ser posible agregar nuevos algoritmos sin modificar el TAD base de forma innecesaria.
RNF-09	Consistencia visual	Se deben mantener colores, espaciado y estilo institucional.
RNF-10	Claridad didáctica	Cada algoritmo debe explicar su lógica en términos comprensibles para estudiantes.

9. Casos de uso

9.1 Diagrama de casos de uso

```
@startuml
left to right direction
actor Estudiante
actor Docente

rectangle "Módulo de Grafos" {
  usecase "Crear grafo" as UC1
  usecase "Insertar vértice" as UC2
  usecase "Insertar arista" as UC3
  usecase "Eliminar vértice" as UC4
  usecase "Eliminar arista" as UC5
  usecase "Ejecutar BFS" as UC6
  usecase "Ejecutar DFS" as UC7
  usecase "Ejecutar Dijkstra" as UC8
  usecase "Ejecutar Bellman-Ford" as UC9
  usecase "Ejecutar Prim" as UC10
  usecase "Ejecutar Kruskal" as UC11
  usecase "Ver código" as UC12
  usecase "Ver explicación" as UC13
  usecase "Consultar complejidad" as UC14
}

Estudiante --> UC1
Estudiante --> UC2
Estudiante --> UC3
Estudiante --> UC4
Estudiante --> UC5
Estudiante --> UC6
Estudiante --> UC7
Estudiante --> UC8
Estudiante --> UC9
Estudiante --> UC10
Estudiante --> UC11
Estudiante --> UC12
Estudiante --> UC13
Estudiante --> UC14

Docente --> UC6
Docente --> UC7
Docente --> UC8
Docente --> UC9
Docente --> UC10
Docente --> UC11
Docente --> UC12
Docente --> UC13
```

```
Docente --> UC14
@enduml
```

9.2 CU-01 — Crear grafo

Campo	Descripción
Nombre	Crear grafo
Actor	Estudiante / Docente
Descripción	Permite inicializar un grafo dirigido o no dirigido.
Precondición	La aplicación está abierta.
Flujo principal	El usuario selecciona el tipo de grafo. El sistema invoca `grafo_crear(dirigido)`. Se inicializa la vista vacía.
Flujo alternativo	Si ya existe un grafo, el sistema debe pedir reinicio o destruir el grafo anterior.
Postcondición	Existe un grafo activo listo para recibir vértices y aristas.

9.3 CU-02 — Insertar vértice

Campo	Descripción
Nombre	Insertar vértice
Actor	Estudiante
Descripción	Agrega un nuevo vértice con identificador entero.
Precondición	Debe existir un grafo activo.
Flujo principal	El usuario ingresa el valor del vértice. El sistema llama `grafo_insertar_vertice`. La vista dibuja el nuevo nodo.
Flujo alternativo	Si el vértice ya existe, se muestra mensaje de duplicado.
Postcondición	El grafo aumenta su orden en una unidad.

9.4 CU-03 — Insertar arista

Campo	Descripción
Nombre	Insertar arista
Actor	Estudiante
Descripción	Agrega una arista ponderada entre dos vértices existentes.
Precondición	Deben existir los vértices origen y destino.
Flujo principal	El usuario ingresa origen, destino y peso. El sistema llama `grafo_insertar_arista`. La vista dibuja línea o flecha con peso.

Campo	Descripción
Flujo alternativo	Si algún vértice no existe, se muestra error. Si la arista existe, se muestra duplicado.
Postcondición	El grafo aumenta su tamaño lógico.

9.5 CU-04 — Ejecutar BFS

Campo	Descripción
Nombre	Ejecutar BFS
Actor	Estudiante / Docente
Descripción	Ejecuta búsqueda en amplitud desde un vértice inicial.
Precondición	El grafo debe existir y contener el vértice inicial.
Flujo principal	El usuario ingresa inicio. El sistema llama `grafo_bfs`. La vista resalta el orden de visita.
Flujo alternativo	Si el vértice no existe, se muestra error.
Postcondición	Se muestra el recorrido BFS, código, explicación y complejidad.

9.6 CU-05 — Ejecutar Dijkstra

Campo	Descripción
Nombre	Ejecutar Dijkstra
Actor	Estudiante / Docente
Descripción	Calcula el camino mínimo entre origen y destino con pesos no negativos.
Precondición	El grafo debe existir, los vértices deben existir y no debe haber pesos negativos.
Flujo principal	El usuario ingresa origen y destino. El sistema llama `grafo_dijkstra`. La vista resalta el camino mínimo y muestra costo total.
Flujo alternativo	Si hay pesos negativos, se informa que el algoritmo no aplica.
Postcondición	Se muestra camino, costo, código, explicación y complejidad.

9.7 CU-06 — Ejecutar Kruskal

Campo	Descripción
Nombre	Ejecutar Kruskal
Actor	Estudiante / Docente
Descripción	Calcula el árbol de expansión mínima usando selección global de aristas.

Campo	Descripción
Precondición	El grafo debe ser no dirigido.
Flujo principal	El sistema llama `grafo_kruskal`. La vista resalta las aristas seleccionadas.
Flujo alternativo	Si el grafo es dirigido, se informa que el algoritmo requiere grafo no dirigido.
Postcondición	Se muestra MST, costo total, código, explicación y complejidad.

---

10. Modelo de datos

10.1 TAD Grafo público

El TAD Grafo se define mediante un tipo opaco:

```
typedef struct Grafo Grafo;
```

El usuario no puede acceder directamente a la representación interna del grafo.

10.2 Estados del TAD

```
typedef enum GrafoEstado {
    GRAFO_OK = 0,
    GRAFO_ERROR_NULO,
    GRAFO_ERROR_MEMORIA,
    GRAFO_ERROR_NO_EXISTE,
    GRAFO_ERROR_YA_EXISTE,
    GRAFO_ERROR_PESO_NEGATIVO,
    GRAFO_ERROR_CICLO_NEGATIVO
} GrafoEstado;
```

Estado	Significado
`GRAFO_OK`	Operación finalizada correctamente.
`GRAFO_ERROR_NULO`	Se recibió un puntero nulo.
`GRAFO_ERROR_MEMORIA`	No fue posible reservar memoria dinámica.
`GRAFO_ERROR_NO_EXISTE`	Vértice o arista inexistente.
`GRAFO_ERROR_YA_EXISTE`	Vértice o arista duplicada.
`GRAFO_ERROR_PESO_NEGATIVO`	El algoritmo no acepta pesos negativos.
`GRAFO_ERROR_CICLO_NEGATIVO`	Bellman-Ford detectó ciclo negativo.

10.3 Arista pública

```
typedef struct GrafoArista {
    int origen;
    int destino;
    int peso;
} GrafoArista;
```

10.4 Resultado de recorrido

```
typedef struct GrafoRecorrido {
    int *vertices;
    size_t cantidad;
    GrafoEstado estado;
} GrafoRecorrido;
```

10.5 Resultado de camino o árbol de expansión

```
typedef struct GrafoCamino {
    GrafoArista *aristas;
    size_t cantidad;
    int costo_total;
    bool existe;
    GrafoEstado estado;
} GrafoCamino;
```

10.6 Estado visual del módulo

```
typedef enum {
    GRAFO_ALG_NINGUNO = 0,
    GRAFO_ALG_BFS,
    GRAFO_ALG_DFS,
    GRAFO_ALG_DIJKSTRA,
    GRAFO_ALG_BELLMAN_FORD,
    GRAFO_ALG_PRIM,
    GRAFO_ALG_KRUSKAL
} GrafoAlgoritmoActivo;

typedef struct {
    int vertice;
    Vector2 posicion;
    bool resaltado;
    bool visitado;
} GrafoVerticeVisual;

typedef struct {
    int origen;
```

```
    int destino;
    int peso;
    bool resaltada;
    bool relajada;
} GrafoAristaVisual;

typedef struct {
    Grafo *grafo;
    bool dirigido;
    GrafoAlgoritmoActivo algoritmo_activo;
    int origen_actual;
    int destino_actual;
    int peso_actual;
    char mensaje[256];
    char complejidad[256];
    char codigo_actual[4096];
} GrafoState;
```

---

11. API pública del TAD Grafo

11.1 Creación y destrucción

```
Grafo *grafo_crear(bool dirigido);
void grafo_destruir(Grafo **grafo);
bool grafo_es_dirigido(const Grafo *grafo);
```

11.2 Operaciones estructurales

```
GrafoEstado grafo_insertar_vertice(Grafo *grafo, int vertice);
GrafoEstado grafo_insertar_arista(Grafo *grafo, int origen, int destino, int
peso);
GrafoEstado grafo_eliminar_vertice(Grafo *grafo, int vertice);
GrafoEstado grafo_eliminar_arista(Grafo *grafo, int origen, int destino);
```

11.3 Consultas

```
bool grafo_existe_vertice(const Grafo *grafo, int vertice);
bool grafo_existe_arista(const Grafo *grafo, int origen, int destino);
GrafoEstado grafo_obtener_peso(const Grafo *grafo, int origen, int destino,
int *peso);
size_t grafo_orden(const Grafo *grafo);
size_t grafo_tamano(const Grafo *grafo);
GrafoEstado grafo_grado_salida(const Grafo *grafo, int vertice, size_t
*grado);
```

```
GrafoEstado grafo_grado_entrada(const Grafo *grafo, int vertice, size_t
*grado);
```

11.4 Copias públicas de datos

```
GrafoEstado grafo_obtener_vertices(const Grafo *grafo, int **vertices,
size_t *cantidad);
GrafoEstado grafo_obtener_aristas(const Grafo *grafo, GrafoArista **aristas,
size_t *cantidad);
GrafoEstado grafo_sucesores(const Grafo *grafo, int vertice, int
**sucesores, size_t *cantidad);
GrafoEstado grafo_predecesores(const Grafo *grafo, int vertice, int
**predecesores, size_t *cantidad);
```

11.5 Algoritmos

```
GrafoRecorrido grafo_bfs(const Grafo *grafo, int inicio);
GrafoRecorrido grafo_dfs(const Grafo *grafo, int inicio);
GrafoCamino grafo_dijkstra(const Grafo *grafo, int origen, int destino);
GrafoCamino grafo_bellman_ford(const Grafo *grafo, int origen, int destino);
GrafoCamino grafo_prim(const Grafo *grafo, int inicio);
GrafoCamino grafo_kruskal(const Grafo *grafo);
```

11.6 Liberación de resultados

```
void grafo_liberar_recorrido(GrafoRecorrido *recorrido);
void grafo_liberar_camino(GrafoCamino *camino);
const char *grafo_estado_cadena(GrafoEstado estado);
```

---

12. Contratos de interfaz UI/UX

12.1 Distribución de pantalla propuesta

```
+-------------------------------------------------------------------------------
+
| Logo UTP          Visualizador de Grafos — Estructuras de Datos       Logo
ISC |
+-------------------------------------------------------------------------------
+
| Menú lateral |       Panel de grafo              | Panel de código
C              |
|              |  vértices, aristas, pesos         | algoritmo u operación
activa   |
|              |
|                                  |
```

```
|
|-----------------------------------|-------------------------------|
|              |  Panel de pasos / explicación     | Panel de
resultados            |
+-------------------------------------------------------------------------------
+
| Estado: orden, tamaño, tipo de grafo, algoritmo
activo                          |
+-------------------------------------------------------------------------------
+
```

12.2 Pantallas del módulo

Pantalla	Descripción
`PANTALLA_GRAFO_INICIO`	Permite seleccionar grafo dirigido o no dirigido.
`PANTALLA_GRAFO_EDICION`	Permite insertar y eliminar vértices y aristas.
`PANTALLA_GRAFO_RECORRIDOS`	Ejecuta BFS y DFS.
`PANTALLA_GRAFO_CAMINOS`	Ejecuta Dijkstra y Bellman-Ford.
`PANTALLA_GRAFO_MST`	Ejecuta Prim y Kruskal.
`PANTALLA_GRAFO_AYUDA`	Explica el uso del módulo.

12.3 Contratos de eventos

Pantalla	Evento	Acción	Destino
Inicio	Clic en “Grafo dirigido”	`grafo_crear(true)`	Edición
Inicio	Clic en “Grafo no dirigido”	`grafo_crear(false)`	Edición
Edición	Botón “Agregar vértice”	`grafo_insertar_vertice`	Edición actualizada
Edición	Botón “Agregar arista”	`grafo_insertar_arista`	Edición actualizada
Edición	Botón “Eliminar vértice”	`grafo_eliminar_vertice`	Edición actualizada
Edición	Botón “Eliminar arista”	`grafo_eliminar_arista`	Edición actualizada
Recorridos	Botón “BFS”	`grafo_bfs`	Resultado recorrido
Recorridos	Botón “DFS”	`grafo_dfs`	Resultado recorrido
Caminos	Botón “Dijkstra”	`grafo_dijkstra`	Resultado camino mínimo

Pantalla	Evento	Acción	Destino
Caminos	Botón “Bellman-Ford”	`grafo_bellman_ford`	Resultado camino mínimo
MST	Botón “Prim”	`grafo_prim`	Resultado MST
MST	Botón “Kruskal”	`grafo_kruskal`	Resultado MST
Cualquier pantalla	Botón “Limpiar”	destruir y crear nuevo grafo	Inicio o Edición

13. Reglas de negocio

ID	Regla
RN-01	No se pueden insertar aristas entre vértices inexistentes.
RN-02	No se pueden insertar vértices duplicados.
RN-03	No se pueden insertar aristas duplicadas.
RN-04	En grafos no dirigidos, una arista se representa lógicamente una sola vez aunque internamente pueda almacenarse en ambos sentidos.
RN-05	Dijkstra no debe ejecutarse si existe al menos un peso negativo.
RN-06	Bellman-Ford debe informar si existe ciclo negativo.
RN-07	Prim solo debe ejecutarse sobre grafos no dirigidos.
RN-08	Kruskal solo debe ejecutarse sobre grafos no dirigidos.
RN-09	Los resultados dinámicos de BFS, DFS, caminos y MST deben liberarse después de usarse.
RN-10	El panel visual no debe modificar directamente la representación interna del grafo.
RN-11	Todo algoritmo ejecutado debe actualizar visualización, código, explicación y complejidad.
RN-12	Si no existe solución, el sistema debe mostrar “No existe camino” o “No existe árbol de expansión”.

14. Especificación de algoritmos

14.1 Insertar vértice

Campo	Descripción
Entrada	Grafo y valor entero del vértice.
Salida	Grafo con nuevo vértice.
Complejidad temporal	`O(V)` por verificación de existencia en lista enlazada.

Campo	Descripción
Complejidad espacial	`O(1)` por nuevo nodo.

Pseudocódigo:

```
si grafo es NULL, retornar error
si vértice ya existe, retornar duplicado
crear nodo de vértice
insertar al inicio de la lista de vértices
incrementar orden
retornar OK
```

14.2 Insertar arista

Campo	Descripción
Entrada	Grafo, origen, destino, peso.
Salida	Grafo con nueva arista ponderada.
Complejidad temporal	`O(V + E)` por validaciones y búsqueda de arista.
Complejidad espacial	`O(1)` en grafo dirigido; `O(2)` físico en grafo no dirigido.

Pseudocódigo:

```
validar grafo
validar existencia de origen y destino
validar que la arista no exista
insertar arista origen -> destino
si el grafo no es dirigido y no es lazo
    insertar arista destino -> origen
retornar OK
```

14.3 BFS

Campo	Descripción
Entrada	Grafo e inicio.
Salida	Orden de visita en amplitud.
Complejidad temporal	`O(V + E)` en representación por listas de adyacencia; en la implementación actual puede aumentar por búsquedas lineales auxiliares.
Complejidad espacial	`O(V)`

Pseudocódigo:

```
marcar inicio como visitado
encolar inicio
mientras la cola no esté vacía
    desencolar actual
    agregar actual al recorrido
    para cada sucesor de actual
        si no está visitado
            marcar visitado
            encolar sucesor
retornar recorrido
```

14.4 DFS

Campo	Descripción
Entrada	Grafo e inicio.
Salida	Orden de visita en profundidad.
Complejidad temporal	`O(V + E)` en representación por listas de adyacencia; en la implementación actual puede aumentar por búsquedas lineales auxiliares.
Complejidad espacial	`O(V)` por arreglos y pila de recursión.

Pseudocódigo:

```
DFS(actual):
    marcar actual como visitado
    agregar actual al recorrido
    para cada sucesor de actual
        si no está visitado
            DFS(sucesor)
```

14.5 Dijkstra

Campo	Descripción
Entrada	Grafo, origen y destino.
Salida	Camino mínimo y costo total.
Restricción	No acepta pesos negativos.
Complejidad temporal	`O(V^2 + E·V)` en la implementación actual con selección lineal e índices lineales.
Complejidad espacial	`O(V)`

Pseudocódigo:

```
inicializar distancias en infinito
inicializar predecesores en -1
dist[origen] = 0
repetir V veces
    seleccionar vértice no visitado con menor distancia
    marcarlo como visitado
    para cada arista saliente
        relajar distancia si mejora
construir camino usando predecesores
```

14.6 Bellman-Ford

Campo	Descripción
Entrada	Grafo, origen y destino.
Salida	Camino mínimo y costo total.
Restricción	Permite pesos negativos, pero detecta ciclos negativos.
Complejidad temporal	`O(V·E·V)` en la implementación actual por búsqueda lineal de índices; teóricamente `O(VE)`.
Complejidad espacial	`O(V)`

Pseudocódigo:

```
inicializar distancias en infinito
inicializar predecesores en -1
dist[origen] = 0
repetir V - 1 veces
    para cada arista u -> v
        si dist[u] + peso < dist[v]
            actualizar dist[v]
            actualizar predecesor[v]
verificar nuevamente todas las aristas
si alguna distancia mejora
    reportar ciclo negativo
construir camino
```

14.7 Prim

Campo	Descripción
Entrada	Grafo no dirigido e inicio.
Salida	Árbol de expansión mínima.
Restricción	Requiere grafo no dirigido.

Campo	Descripción
Complejidad temporal	`O(V^2 + E·V)` en la implementación actual.
Complejidad espacial	`O(V)`

Pseudocódigo:

```
inicializar claves en infinito
clave[inicio] = 0
mientras existan vértices fuera del MST
    seleccionar vértice con menor clave
    agregarlo al MST
    para cada arista saliente
        si el destino no está en MST y el peso mejora la clave
            actualizar clave
            actualizar padre
construir conjunto de aristas del MST
```

14.8 Kruskal

Campo	Descripción
Entrada	Grafo no dirigido.
Salida	Árbol de expansión mínima.
Restricción	Requiere grafo no dirigido.
Complejidad temporal	`O(E log E + E α(V))`, más costos de búsqueda lineal de índices en la implementación actual.
Complejidad espacial	`O(V + E)`

Pseudocódigo:

```
obtener aristas del grafo
ordenar aristas por peso
crear conjuntos disjuntos para cada vértice
para cada arista en orden ascendente
    si origen y destino pertenecen a conjuntos diferentes
        agregar arista al MST
        unir conjuntos
retornar MST
```

---

15. Diseño visual del grafo

15.1 Representación de vértices

Cada vértice debe dibujarse como un círculo:

```
(10)
```

Estados visuales:

Estado	Representación sugerida
Normal	Círculo azul o gris.
Visitado	Círculo resaltado.
Vértice inicial	Borde destacado.
Vértice destino	Borde doble.
Vértice actual	Color temporal de análisis.

15.2 Representación de aristas

Grafo no dirigido:

```
(10) -------- 5 -------- (20)
```

Grafo dirigido:

```
(10) ---- 5 ----> (20)
```

Estados visuales:

Estado	Representación sugerida
Normal	Línea gris.
Relajada	Línea resaltada temporalmente.
Camino mínimo	Línea destacada.
MST	Línea gruesa o color especial.
Error/ciclo negativo	Línea o nodo en advertencia.

15.3 Distribución inicial de vértices

Para la primera versión se propone una distribución circular automática:

```
            (1)
      (6)         (2)

      (5)         (3)
            (4)
```

La función `grafo_layout_calcular_circular` recibirá la cantidad de vértices y asignará posiciones alrededor del centro del panel.

---

16. Visor de código ejecutado

16.1 Propósito

El panel de código debe mostrar fragmentos relevantes del archivo `grafo.c`, no necesariamente todo el archivo. Cada operación debe tener un fragmento representativo.

16.2 Estructura sugerida

```c
typedef struct {
    GrafoAlgoritmoActivo algoritmo;
    const char *titulo;
    const char *codigo;
    const char *explicacion;
    const char *complejidad;
} GrafoSnippet;
```

16.3 Ejemplo de fragmento para BFS

```c
static const char *CODIGO_GRAFO_BFS =
"GrafoRecorrido grafo_bfs(const Grafo *grafo, int inicio) {\n"
"    /* validar grafo e inicio */\n"
"    /* crear arreglo de visitados */\n"
"    /* encolar vértice inicial */\n"
"    while (!cola_vacia_i(&cola)) {\n"
"        cola_desencolar_i(&cola, &actual);\n"
"        r.vertices[r.cantidad++] = actual;\n"
"        /* recorrer sucesores no visitados */\n"
"    }\n"
"    return r;\n"
"}";
```

---

17. Trazas didácticas

17.1 Trazas para BFS

Paso	Mensaje
1	Se valida que el grafo exista y que el vértice inicial esté registrado.
2	Se crea un arreglo de visitados.
3	Se marca el vértice inicial como visitado.
4	Se encola el vértice inicial.
5	Mientras la cola no esté vacía, se extrae el siguiente vértice.
6	Se visitan sus sucesores no visitados.
7	El recorrido final representa una exploración por niveles.

17.2 Trazas para Dijkstra

Paso	Mensaje
1	Se verifica que no existan pesos negativos.
2	Se inicializan distancias en infinito.
3	La distancia del origen se fija en cero.
4	Se selecciona el vértice no visitado con menor distancia.
5	Se relajan las aristas salientes del vértice seleccionado.
6	Se actualizan predecesores cuando aparece una ruta más corta.
7	Se reconstruye el camino desde destino hacia origen.

17.3 Trazas para Kruskal

Paso	Mensaje
1	Se obtienen todas las aristas lógicas del grafo.
2	Se ordenan las aristas por peso ascendente.
3	Cada vértice inicia en su propio conjunto.
4	Se evalúa cada arista ordenada.
5	Si la arista no forma ciclo, se agrega al MST.
6	Se unen los conjuntos de sus vértices.
7	El proceso termina cuando se seleccionan V - 1 aristas.

18. Diagramas de arquitectura

18.1 Diagrama de componentes

```
@startuml
skinparam componentStyle rectangle

package "Presentación Raylib" {
  [main.c]
  [ui.c]
  [grafo_view.c]
}

package "Control" {
  [grafo_controller.c]
  [grafo_state.c]
}

package "Didáctica" {
  [grafo_code_viewer.c]
  [grafo_trace.c]
}

package "Dominio" {
  [grafo.h]
  [grafo.c]
}

package "Layout" {
  [grafo_layout.c]
}

[main.c] --> [ui.c]
[main.c] --> [grafo_controller.c]
[grafo_controller.c] --> [grafo.c]
[grafo_controller.c] --> [grafo_state.c]
[grafo_view.c] --> [grafo_layout.c]
[grafo_view.c] --> [grafo.c]
[grafo_view.c] --> [grafo_code_viewer.c]
[grafo_view.c] --> [grafo_trace.c]
@enduml
```

18.2 Diagrama de paquetes

```
@startuml
package "app" {
  class MainLoop
}
```

```
package "ui" {
  class UI
  class Header
  class Menu
  class Panel
}

package "graph_module" {
  class GrafoController
  class GrafoState
  class GrafoView
  class GrafoLayout
}

package "graph_domain" {
  class Grafo
  class GrafoArista
  class GrafoRecorrido
  class GrafoCamino
}

package "didactic" {
  class GrafoCodeViewer
  class GrafoTrace
}

app --> ui
app --> graph_module
graph_module --> graph_domain
graph_module --> didactic
@enduml
```

18.3 Diagrama de clases lógico

```
@startuml
class GrafoState {
  Grafo *grafo
  bool dirigido
  GrafoAlgoritmoActivo algoritmo_activo
  int origen_actual
  int destino_actual
  int peso_actual
  char mensaje[256]
  char complejidad[256]
}

class Grafo {
  <<opaque>>
}
```

```
class GrafoArista {
  int origen
  int destino
  int peso
}

class GrafoRecorrido {
  int *vertices
  size_t cantidad
  GrafoEstado estado
}

class GrafoCamino {
  GrafoArista *aristas
  size_t cantidad
  int costo_total
  bool existe
  GrafoEstado estado
}

class GrafoController {
  crear_grafo()
  insertar_vertice()
  insertar_arista()
  ejecutar_algoritmo()
}

class GrafoView {
  dibujar_grafo()
  dibujar_vertices()
  dibujar_aristas()
  resaltar_resultado()
}

class GrafoCodeViewer {
  cargar_snippet()
  dibujar_codigo()
}

class GrafoTrace {
  generar_pasos()
  dibujar_traza()
}

GrafoState *-- Grafo
GrafoController --> GrafoState
GrafoController --> Grafo
GrafoView --> GrafoState
GrafoView --> Grafo
GrafoView --> GrafoCodeViewer
GrafoView --> GrafoTrace
```

```
GrafoCamino *-- GrafoArista
@enduml
```

---

19. Diagramas de secuencia

19.1 Insertar vértice

```
@startuml
actor Usuario
participant "UI Raylib" as UI
participant "GrafoController" as Controller
participant "GrafoState" as State
participant "TAD Grafo" as Grafo
participant "GrafoView" as View

Usuario -> UI: Ingresa vértice y presiona Agregar
UI -> Controller: insertar_vertice(valor)
Controller -> Grafo: grafo_insertar_vertice(grafo, valor)
Grafo --> Controller: GrafoEstado
Controller -> State: actualizar mensaje y código
Controller -> View: solicitar redibujado
View -> Grafo: grafo_obtener_vertices()
View -> UI: dibujar vértices
@enduml
```

19.2 Ejecutar BFS

```
@startuml
actor Usuario
participant "UI Raylib" as UI
participant "GrafoController" as Controller
participant "TAD Grafo" as Grafo
participant "GrafoTrace" as Trace
participant "GrafoCodeViewer" as Code
participant "GrafoView" as View

Usuario -> UI: Ingresa inicio y presiona BFS
UI -> Controller: ejecutar_bfs(inicio)
Controller -> Grafo: grafo_bfs(grafo, inicio)
Grafo --> Controller: GrafoRecorrido
Controller -> Trace: generar_traza_bfs()
Controller -> Code: cargar_codigo_bfs()
Controller -> View: resaltar recorrido
View -> UI: redibujar grafo visitado
Controller -> Grafo: grafo_liberar_recorrido()
@enduml
```

19.3 Ejecutar Dijkstra

```
@startuml
actor Usuario
participant "UI Raylib" as UI
participant "GrafoController" as Controller
participant "TAD Grafo" as Grafo
participant "GrafoTrace" as Trace
participant "GrafoCodeViewer" as Code
participant "GrafoView" as View

Usuario -> UI: Ingresa origen/destino y presiona Dijkstra
UI -> Controller: ejecutar_dijkstra(origen, destino)
Controller -> Grafo: grafo_dijkstra(grafo, origen, destino)
Grafo --> Controller: GrafoCamino
Controller -> Trace: generar_traza_dijkstra()
Controller -> Code: cargar_codigo_dijkstra()
Controller -> View: resaltar camino mínimo
View -> UI: dibujar camino y costo
Controller -> Grafo: grafo_liberar_camino()
@enduml
```

---

20. Trazabilidad

RF	Caso de uso	Módulo	Prueba
RF-01	CU-01	`grafo_controller.c`	PT-01
RF-02	CU-02	`grafo.c` / `grafo_view.c`	PT-02
RF-03	CU-03	`grafo.c` / `grafo_view.c`	PT-03
RF-04	Eliminar vértice	`grafo.c`	PT-04
RF-05	Eliminar arista	`grafo.c`	PT-05
RF-12	CU-04	`grafo_bfs`	PT-08
RF-13	DFS	`grafo_dfs`	PT-09
RF-14	CU-05	`grafo_dijkstra`	PT-10
RF-15	Bellman-Ford	`grafo_bellman_ford`	PT-11
RF-16	Prim	`grafo_prim`	PT-12
RF-17	CU-06	`grafo_kruskal`	PT-13
RF-18	Ver código	`grafo_code_viewer.c`	PT-14
RF-19	Ver explicación	`grafo_trace.c`	PT-15
RF-20	Ver complejidad	`grafo_trace.c`	PT-16

21. Plan de pruebas

ID	Descripción	Entrada	Resultado esperado
PT-01	Crear grafo dirigido	dirigido = true	Grafo activo dirigido.
PT-02	Insertar vértice	10	Vértice 10 visible.
PT-03	Insertar arista	10, 20, 5	Arista 10 -> 20 con peso 5.
PT-04	Insertar arista con vértice inexistente	10, 99, 3	Error `GRAFO_ERROR_NO_EXISTE`.
PT-05	Insertar vértice duplicado	10 dos veces	Error `GRAFO_ERROR_YA_EXISTE`.
PT-06	Eliminar vértice	10	Se elimina vértice y aristas incidentes.
PT-07	Eliminar arista	10, 20	Se elimina la arista.
PT-08	BFS	inicio 1	Se muestra orden por niveles.
PT-09	DFS	inicio 1	Se muestra orden en profundidad.
PT-10	Dijkstra válido	origen 1, destino 5	Se muestra camino mínimo y costo.
PT-11	Dijkstra con peso negativo	arista con peso -2	Error por peso negativo.
PT-12	Bellman-Ford con pesos negativos sin ciclo	origen 1, destino 5	Camino mínimo calculado.
PT-13	Bellman-Ford con ciclo negativo	ciclo con suma negativa	Error de ciclo negativo.
PT-14	Prim en grafo no dirigido	inicio 1	Se muestra MST.
PT-15	Prim en grafo dirigido	dirigido = true	Error por algoritmo no aplicable.
PT-16	Kruskal en grafo no dirigido	grafo ponderado	Se muestra MST.
PT-17	Kruskal en grafo desconectado	dos componentes	Se informa que no existe MST completo.
PT-18	Ver código BFS	ejecutar BFS	Panel muestra fragmento de `grafo_bfs`.
PT-19	Ver explicación Dijkstra	ejecutar Dijkstra	Panel muestra pasos de relajación.
PT-20	Cierre de módulo	salir	Memoria del grafo y resultados liberada.

22. Criterios de aceptación

ID	Criterio
CA-01	El módulo permite crear grafos dirigidos y no dirigidos.
CA-02	El módulo permite insertar vértices y aristas ponderadas.
CA-03	El módulo impide insertar aristas con vértices inexistentes.
CA-04	El módulo dibuja vértices, aristas, pesos y flechas correctamente.
CA-05	BFS muestra el orden de visita y resalta vértices visitados.
CA-06	DFS muestra el orden de visita y resalta vértices visitados.
CA-07	Dijkstra calcula camino mínimo cuando los pesos son no negativos.
CA-08	Dijkstra rechaza grafos con pesos negativos.
CA-09	Bellman-Ford detecta ciclos negativos.
CA-10	Prim solo se ejecuta en grafos no dirigidos.
CA-11	Kruskal solo se ejecuta en grafos no dirigidos.
CA-12	Los caminos mínimos y MST se resaltan visualmente.
CA-13	Cada algoritmo muestra código, explicación y complejidad.
CA-14	El módulo usa únicamente la API pública de `grafo.h`.
CA-15	Los resultados dinámicos se liberan correctamente.
CA-16	El código fuente está documentado con Doxygen.
CA-17	El módulo compila con GCC y Raylib.

---

23. Documentación Doxygen

23.1 Encabezado de archivo

```
/**
 * @file grafo_view.c
 * @brief Vista gráfica del módulo de grafos usando Raylib.
 *
 * Este archivo contiene las funciones responsables de dibujar vértices,
 * aristas, pesos, recorridos y resultados de algoritmos de grafos.
 * No modifica directamente la estructura interna del TAD Grafo.
 *
 * @author Francisco Alejandro Medina Aguirre
 * @date 2026
 */
```

23.2 Documentación de función pública

```c
/**
 * @brief Dibuja el grafo activo en el panel principal.
 *
 * Obtiene una copia pública de vértices y aristas mediante la API del TAD
 * Grafo y los representa visualmente usando Raylib.
 *
 * @param[in] state Estado visual del módulo de grafos.
 * @param[in] bounds Rectángulo del panel donde se dibuja el grafo.
 *
 * @pre state debe estar inicializado.
 * @post El grafo se representa gráficamente sin modificar el TAD.
 * @note Complejidad temporal: O(V + E) más el costo del layout.
 */
void grafo_view_dibujar(const GrafoState *state, Rectangle bounds);
```

---

24. Estrategia de implementación incremental

Fase 1 — Integración base del TAD

- Ubicar `grafo.h` en `include/`.
- Ubicar `grafo.c` en `src/`.
- Compilar pruebas de consola para creación, inserción y eliminación.
- Verificar liberación de memoria.

Fase 2 — Vista inicial del grafo

- Crear `grafo_state.h/c`.
- Crear `grafo_view.h/c`.
- Implementar dibujo de vértices.
- Implementar dibujo de aristas y pesos.
- Implementar layout circular.

Fase 3 — Control de operaciones básicas

- Crear `grafo_controller.h/c`.
- Implementar crear grafo, insertar vértice, insertar arista.
- Implementar eliminar vértice y eliminar arista.
- Mostrar mensajes del estado del TAD.

Fase 4 — Recorridos

- Integrar BFS.
- Integrar DFS.
- Resaltar orden de visita.
- Mostrar código y explicación.

Fase 5 — Caminos mínimos

- Integrar Dijkstra.
- Integrar Bellman-Ford.
- Resaltar camino y costo total.
- Mostrar restricciones y errores.

Fase 6 — Árboles de expansión mínima

- Integrar Prim.
- Integrar Kruskal.
- Resaltar aristas seleccionadas.
- Mostrar costo total y si existe MST completo.

Fase 7 — Documentación y pruebas

- Completar Doxygen.
- Crear README del módulo.
- Crear guía de laboratorio.
- Ejecutar pruebas funcionales.

---

25. Comando de compilación sugerido

En Windows con MSYS2 UCRT64 y Raylib instalado:

```
gcc src/main.c src/ui.c src/grafo.c src/grafo_state.c src/grafo_view.c
    src/grafo_layout.c src/grafo_controller.c src/grafo_code_viewer.c
    src/grafo_trace.c
    -Iinclude -o visualizador_grafos.exe -lraylib -lopengl32 -lgdi32 -lwinmm
```

Con `Makefile`:

```
make run-grafos
```

---

26. Riesgos técnicos

Riesgo	Impacto	Mitigación
Acoplar Raylib dentro del TAD	Alto	Mantener `grafo.c` libre de dependencias gráficas.
No liberar recorridos o caminos	Alto	Usar siempre `grafo_liberar_recorrido` y `grafo_liberar_camino`.
Visualización confusa en grafos grandes	Medio	Limitar versión inicial a grafos pequeños y medianos.

Riesgo	Impacto	Mitigación
Cruce excesivo de aristas	Medio	Usar layout circular y permitir ajuste posterior.
Dijkstra con pesos negativos	Alto	Validar y mostrar error didáctico.
Prim/Kruskal en grafo dirigido	Alto	Validar tipo de grafo antes de ejecutar.
Errores por vértices duplicados	Medio	Mostrar mensajes claros según `GrafoEstado`.
Animaciones complejas prematuras	Medio	Priorizar visualización estática clara en primera versión.

---

27. Recomendaciones pedagógicas de uso

1. Iniciar con grafos pequeños de 4 a 6 vértices.
2. Comparar BFS y DFS usando el mismo grafo.
3. Explicar Dijkstra mostrando por qué no acepta pesos negativos.
4. Usar Bellman-Ford para introducir relajación repetida y ciclos negativos.
5. Comparar Prim y Kruskal en el mismo grafo no dirigido.
6. Pedir a los estudiantes predecir el recorrido o el MST antes de ejecutar el algoritmo.
7. Relacionar cada paso visual con el fragmento de código mostrado.
8. Exigir análisis de complejidad como parte del laboratorio.

---

28. Definición de terminado

El módulo de grafos se considerará terminado cuando:

- Compile correctamente con GCC y Raylib.
- Permita crear grafos dirigidos y no dirigidos.
- Permita insertar y eliminar vértices y aristas.
- Dibuje correctamente vértices, aristas, pesos y flechas.
- Ejecute BFS, DFS, Dijkstra, Bellman-Ford, Prim y Kruskal.
- Muestre código, explicación y complejidad por algoritmo.
- Resalte recorridos, caminos mínimos y MST.
- Maneje errores del TAD sin cerrar la aplicación.
- Libere toda la memoria dinámica.
- Mantenga separación entre TAD, vista, controlador y trazas didácticas.
- Incluya documentación Doxygen.
- Incluya pruebas funcionales mínimas.

---

29. Prompt base para agente de IA

```
Actúa como arquitecto de software senior y desarrollador experto en C
estándar, Raylib, estructuras de datos y documentación Doxygen.

Debes implementar el módulo de grafos para un visualizador académico de
estructuras de datos. Usa como base esta especificación SDD. El proyecto debe
respetar separación estricta entre TAD, interfaz gráfica, controlador y
trazas didácticas.

Restricciones obligatorias:
- No modificar innecesariamente la API pública de grafo.h.
- No incluir raylib.h dentro de grafo.c ni grafo.h.
- Usar el TAD Grafo únicamente mediante sus funciones públicas.
- Documentar funciones públicas con Doxygen.
- Validar errores de memoria, punteros nulos y operaciones inválidas.
- Liberar resultados dinámicos con grafo_liberar_recorrido y
grafo_liberar_camino.
- Mostrar visualmente vértices, aristas, pesos, recorridos, caminos mínimos y
MST.
- Mostrar código C, explicación paso a paso y complejidad por operación.

Primero analiza el proyecto actual y propón el plan de integración. Luego
implementa incrementalmente el módulo.
```

---

30. Control de versiones

Versión	Fecha	Cambios
1.0	2026	Documento SDD inicial del módulo de grafos.